Stat 184 Reproducible Data Analysis

Homework 4.3

Jason Kaufmann

2026-04-07

Table of contents

1 Busiest Airports Analysis **2**
1.1 Objective . . . 2
1.2 Table of Descriptive Statistics . . . 2
1.3 Faceted Line Plots by Continent . . . 3

2 Monte Carlo Numerical Integration **4**
2.1 Objective . . . 4
2.2 Protocol for Monte Carlo Integration . . . 4
2.3 Creating the Plots . . . 5
2.4 Interpreting the Plots . . . 5

3 Planning and Prompting GenAI Tools **6**
3.1 A Detailed Plan for Data Wrangling and Visualization . . . 6
3.2 Response and Plot Generated from the Detailed Prompt . . . 7
3.3 Response and Plot from the Generic Prompt . . . 10
3.4 Analyzing Discrepancies in Plot Quality Between a Generic and Detailed Prompt 11

4 Reflection **12**
4.1 Persistence is Everything! . . . 12
4.2 The Importance of Being Detail-Oriented . . . 12

5 Code Appendices **13**
5.1 Personal Code Appendix . . . 13
5.2 GenAI Code Appendix . . . 28

1 Busiest Airports Analysis

1.1 Objective

The main goal of this project was to elucidate how the number of annual passengers have changed at major airlines from 2020 to 2025 and predict how this trend will manifest in future years. The following were all smaller goals that helped us reach this conclusion:

1. Scrape airport data from a Wikipedia page with information on the world's busiest airports.
2. Wrangle the data for each year from 2020 to 2025 and bind these dataframes.
3. Create a polished table of descriptive statistics that shows the minimum, maximum, average, median, and standard deviation for each airport's percent change in annual travelers.
4. Render an informational plot that shows the annual travelers at each airport from 2020 to 2025 and how this trend varies between the North American, European, and Asian airports selected.

Data analysis was conducted for six of the world's busiest airports between 2020 and 2025:

- Beijing Daxing International Airport
- Charles de Gaulle Airport
- Frankfurt Airport
- Guangzhou Baiyun International Airport
- Hartsfield-Jackson Atlanta International Airport
- O'Hare International Airport

1.2 Table of Descriptive Statistics

Table 1 tracks each airport's percentage change in annual travelers between 2020 and 2025. The minimum, maximum, average, median, and standard deviation was computed for each airport (See Table 1)

Table 1: Table of descriptive statistics for major airports' percent change in annual travelers between 2020 and 2025

Airport	Min Annual Percent Change	Max Annual Percent Change	Average Annual Percent Change	Median Annual Percent Change	St Dev Annual Percent Change
Beijing Daxing International Airport	8.4	55.7	29.87	25.5	23.95
Charles de Gaulle Airport	-70.8	119.4	15.07	10.8	60.90
Frankfurt Airport	-73.4	97.2	13.93	12.5	55.10
Guangzhou Baiyun International Airport	-40.4	142.0	24.80	9.5	69.48
Hartsfield–Jackson Atlanta International Airport	-61.2	76.4	8.73	7.5	44.37
O'Hare International Airport	-63.5	75.1	10.08	8.2	44.58

Note: Data sourced from the Airports Council International via Wikipedia

Table 1 shows the impact that the COVID-19 pandemic may have had on 6 major airports and their rebound in recent years. Five of the six airports (all but Beijing Daxing International) had a decrease in annual travelers that was larger than 40%, with some changes being as drastic as Charles de Gaulle's 70.8% decrease between 2019 and 2020. I believe that this phenomenon is most likely due to the travel restrictions and social distancing caused by the COVID-19 pandemic. This caused the data for annual travelers and annual percent change in annual travelers to be left skewed, with the median being much larger than the mean. Another effect of this is an inflation of the maximum annual percent change values for each airport. After a drastic decrease in travelers between 2019 and 2020, all airports saw an extremely large increase in the following years. This variance is reflected in the standard deviation for percent change in annual travelers, which is larger than 44% for 5 of the six airports!

1.3 Faceted Line Plots by Continent

Figure 1 contains three line plots that further illustrate changes in annual travelers over time for the same six airports. The airports were grouped by their continent, which yielded plots for Asia, Europe, and North America. Each plot contained information regarding annual travelers between 2020 and 2025 two airports (See Figure 1).

Figure 1: Graphs of annual travelers over time for six major airports between 2020 and 2025, grouped by their continent

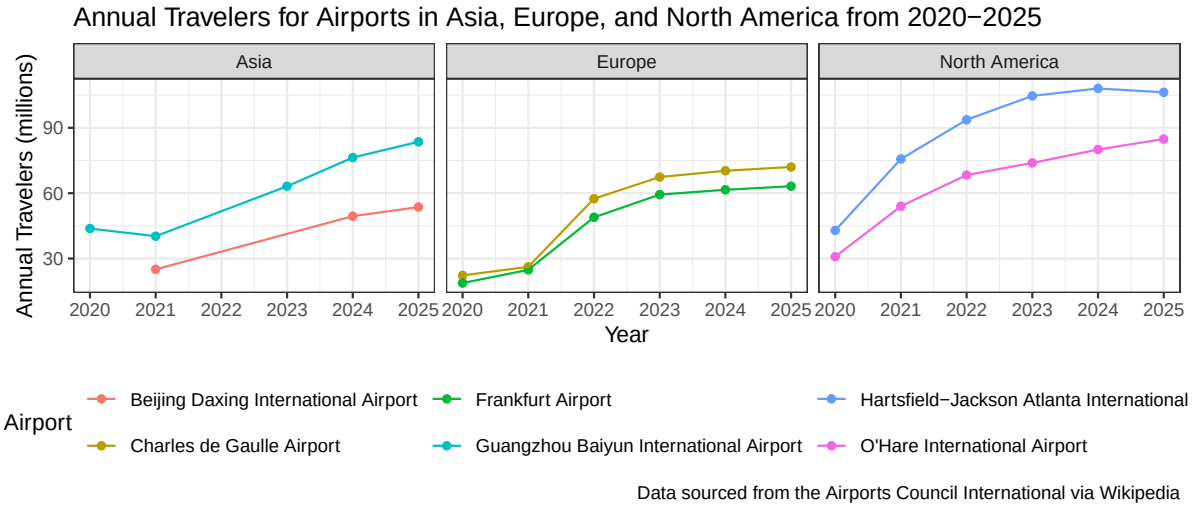


Figure 1 suggests that the rapid increase in each airport's annual travelers after the pandemic is not representative of its future growth. The airports all saw a large increase in annual travelers between 2021 and 2023 that has slowed growth in 2024 and 2025. So, despite extreme maximum and minimum values, the plot suggests that airports will see slower growth in future years as

the effect of COVID-19 lessens. Although the curves for all six airlines have similar shapes, the airports in North America had the most pronounced rebound in recent years, suggesting that COVID-19 may have had a larger impact on American air travel than other nations.

2 Monte Carlo Numerical Integration

2.1 Objective

Monte Carlo Integration utilizes the generation of random ordered pairs to estimate the area under a function. The main goal of this project was to create four small multiples of 10, 100, 1,000, and 10,000 points that integrated the probability density function for the beta distribution. The following were all smaller-scale goals that were achieved along the way:

1. Create a function that generates a random set of ordered pairs depending on the user's preference of the bounds and sample size
2. Apply the probability beta distribution to the set of ordered pairs generated by the function
3. Create a flagging system that will show whether each ordered pair is above or below the output of the beta distribution function. Integrate the probability density function for the beta distribution based the number of points below the function
4. Create four plots that color each ordered pair based on whether it is above or below the function and that show the integration of the probability density function for the beta distribution. ### Random Coordinate Generator Function

2.2 Protocol for Monte Carlo Integration

To use the Monte Carlo Integration method in R, you must create a random coordinate generator function. This function must be able to take upper and lower bounds for both x and y as parameters, as well as the sample size that the user desires. Once this function is created, the user can apply the beta distribution to all x-coordinates that were generated. There must be a flagging system in place such that all y-coordinates are labeled as being either above or below the function. This flagging system should be implemented using both of the following methods:

1. Each point should be labeled as "above" or "below" the function so that the points can be colored based on this attribute.
2. Each point should be assigned a value of 1 if it is above the function and a value of 0 if it is below the function.

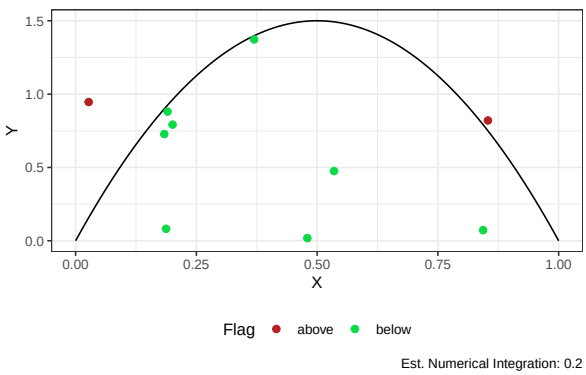
The second method listed above is key for estimating the area under the curve: it allows us to easily determine what proportion of points are below the curve (by taking the average value of this column in the dataframe) without manually counting each point's label.

2.3 Creating the Plots

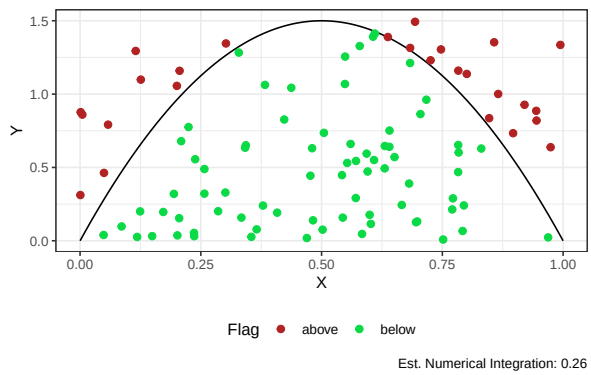
Four plots were created: each with the x-coordinate mapped to the x-axis and y-coordinate mapped to the y-axis. The coordinates that were generated from the previously mentioned function are plotted on the graph and colored based on whether they are above the curve (red) or below it (green). The beta distribution function was also plotted for viewers' reference. Each plot contained a different sample size of coordinates from the coordinate generator function and listed the estimated numerical integration below the plot (See Figure 2).

Figure 2: Monte Carlo integration of the Beta Distribution With Four Unique Sample Sizes

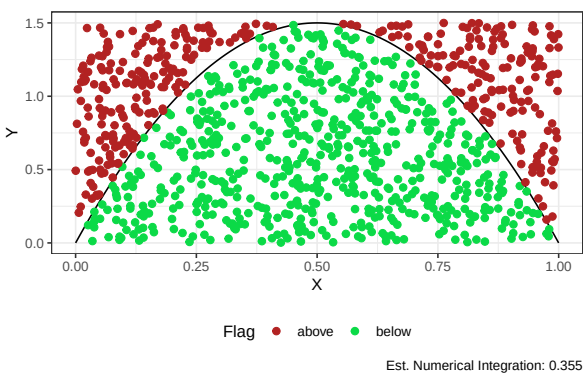
(a) Integration with 10 Points



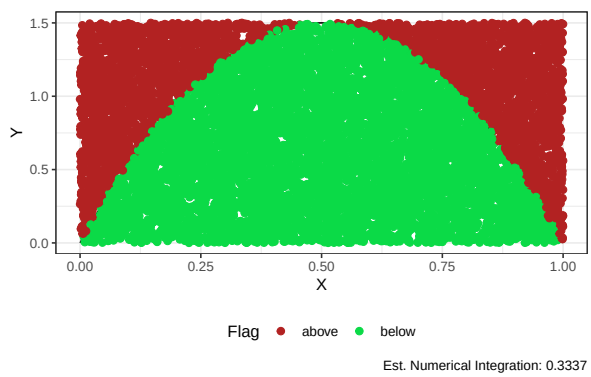
(b) Integration with 100 Points



(c) Integration with 1,000 Points



(d) Integration with 10,000 Points



2.4 Interpreting the Plots

These figures illustrate how the plotting of random ordered pairs can be used to evaluate the area under a curve, and the area under my assigned function is most likely 0.3363. Furthermore, the graph makes it obvious that the estimated numerical integration becomes more precise as the number of plotted points increases, as a larger proportion of the plot becomes populated

with coordinates. Since the estimated area depends on whether these coordinates are flagged as above or below the curve, a larger sample size will produce a more accurate answer for area. The graph appeared to be full when 10,000 points were plotted, so the corresponding estimated numerical integration of 0.3363 will match the actual area under the curve. These small multiples suggest that if someone else replicated this process, they would arrive at the same conclusion as long as they plotted a sufficient number of coordinates from the coordinate generator function.

3 Planning and Prompting GenAI Tools

The goal of this section is to evaluate how the quality of code written by GenAI tools differ based on the quality of the prompt they receive. For this investigation, I focused on a study that tracked women's calcium levels over the course of four years. Each woman was placed in one of two treatment groups: she either participated in a regimented diet/exercise program or did not receive any special treatment. I provided OpenAI's ChatGPT with a detailed prompt of the relevant goals, needs, and necessary steps to create an informative data visualization.

3.1 A Detailed Plan for Data Wrangling and Visualization

Below is the exact prompt that I provided to ChatGPT:

Goal: To wrangle the data from the calcium csv into tidy form and create an informative data visualization that shows how calcium levels between both treatment groups varied over the experiment's duration

Needs:

Nouns: Plan, R Project, R Script, tidyverse, packages, calcium csv, data, columns, scatterplot, framework, mapping, title, treatment group, caption

Verbs: Create, Copy, Fill, Wrangle, Load, Add, Rearrange, Define, Color, Select, Add, Render, Debug, Generate

Steps:

1. Create my plan
2. Create an R Project for this work
3. Create a blank R script
4. Copy and paste the data wrangling template
5. Fill in the template with any relevant information
6. Wrangle the data

    1. Load necessary packages
    2. Load the calcium csv file
    3. Add descriptive column names
    4. Rearrange the data so that each case is a woman in the study. The data in the 5th through 8th columns should be joined with the 1st through 4th columns so that there is one column for each year's data
    5. A new column should be created to identify whether the women were part of the null or experimental groups
    6. Each cell should contain the same number of decimal points
7. Create an informative data visualization that shows the difference in calcium levels between the null treatment and experimental groups over the duration of the experiment
    1. Define the framework and mapping for the plot
    2. Color the points based on their treatment group
    3. Select the appropriate geometry
    4. Add descriptive titles for the plot axes
    5. Add an informative caption that cites the data source
    6. Render the plot
8. Debug if any errors came up
9. Generate Alt Text and a long description for the plot

3.2 Response and Plot Generated from the Detailed Prompt

When provided with a detailed prompt, ChatGPT produced a surprisingly high-quality visual aide (See Figure 3). The response it provided is included verbatim below:

"Good—this is a very solid plan already. I'll tighten it into something you can directly execute in **R (tidyverse style)**.

Refined Plan (What you actually need to do)

Right now your CSV has **two groups side-by-side**:

- y0–y3 → likely **null/control group**
- y0.1–y3.1 → likely **experimental group**

Each row = a woman
Each column = calcium level at a specific year

But this is **wide format**, and you need **tidy (long) format**.

Step 1: Set up your R Project + Script

You already outlined this well:

- Create R Project
- Create script (e.g., calcium_analysis.R)
- Load packages:

library(tidyverse)

Step 2: Load the Data

calcium <- read_csv("calcium.csv")

Step 3: Clean + Rename Columns

Right now your columns look like:

y0, y1, y2, y3, y0.1, y1.1, y2.1, y3.1

Rename them so they clearly indicate:

- **year**
- **treatment group**

Step 4: Add an ID Column

Each woman should be identifiable:

calcium <- calcium %>%

mutate(id = row_number())

Step 5: Pivot to Tidy Format

This is the **most important step**.

calcium_long <- calcium %>%

Step 6: Clean Variables

Make them readable:

calcium_long <- calcium_long %>%

Step 7: Create Visualization

You want:

- comparison over time
- grouped by treatment
- clear trend

Best choice: line plot + points

ggplot(calcium_long, aes(x = year, y = calcium, color = treatment)) +

Why This Works (Important Concept)

- pivot_longer() → makes data tidy
- stat_summary(mean) → shows **group trends instead of noisy individual lines**
- color → distinguishes treatment groups clearly

Step 8: Debug Checklist

If something breaks, check:

- Column names not matching → use colnames(calcium)
- y0.1 issues → must use backticks (y0.1)
- plot looks messy → you forgot stat_summary

Step 9: Alt Text

Short alt text:

Line graph showing average calcium levels over four years for control and experimental groups, with the experimental group generally maintaining higher calcium levels.

Step 10: Long Description

This plot displays the average calcium levels of women over four years in a study comparing a control (null) group and an experimental treatment group. The x-axis represents time in years (0 through 3), and the y-axis shows calcium levels. Each group is represented by a distinct colored line with points marking yearly averages. The experimental group consistently shows slightly higher calcium levels than the control group, suggesting a potential positive effect of the treatment over time.

One Small Critique of Your Original Plan

You said:

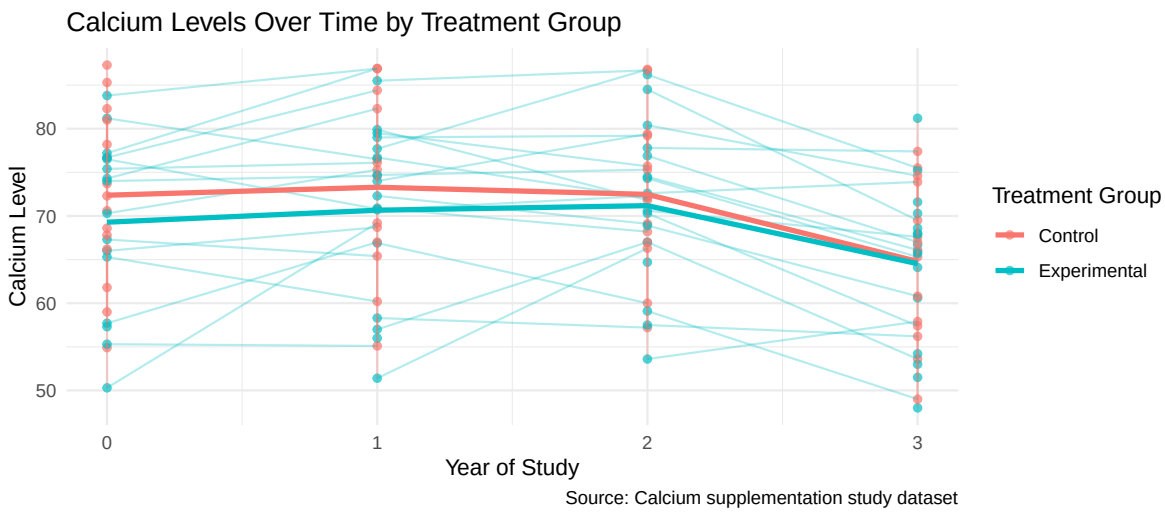
"Each case is a woman"

That's **true initially**, but after tidying:

Each case becomes **one observation per woman per year**

That distinction matters in stats + visualization."

Figure 3: Line plot shows the control group had a higher average calcium level each year in the experiment



3.3 Response and Plot from the Generic Prompt

The plot above stands in stark contrast to the data visualization created by ChatGPT when given a generic prompt (See Figure 4). The exact prompt provided was "Create a data visualization for these data. Calcium.csv" The response is included verbatim below, along with the resulting data plot:

I created a visualization showing the **average value of each column** in your dataset.

What this shows:

- Each point represents the **mean (average)** of one column (y0, y1, y2, y3, etc.).
- You can quickly compare which variables tend to have higher or lower values overall.
- For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create:

- Boxplots (to show spread and outliers)
- Time series plots (if rows represent time/order)
- Correlation heatmaps

- Side-by-side comparisons (e.g., y0 vs y0.1)

Just tell me

Prompt: What is your version information?

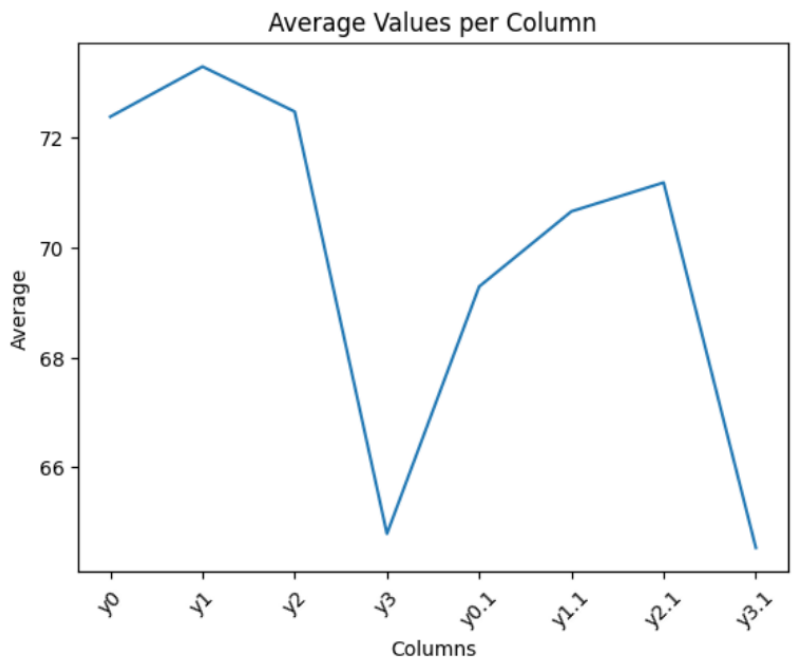
You're chatting with **ChatGPT based on the GPT-5.3 architecture**.

Key details:

- **Model**: GPT-5.3
- **Knowledge cutoff**: August 2025
- **Current date (system)**: March 31, 2026

If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know

Figure 4: The line plot created by ChatGPT when given a generic prompt



3.4 Analyzing Discrepancies in Plot Quality Between a Generic and Detailed Prompt

The importance of providing GenAI tools with detailed prompts can be seen in both the text response it provided and the resulting visual. When provided with a detailed prompt,

ChatGPT addressed me as someone with a coding background and provided me with more of an explanation into what it was doing. However, when given a generic prompt, ChatGPT provided a vague and unhelpful description that provided more empty positive reinforcement than insightful information. The plots tell a similar story: when given a detailed prompt, ChatGPT produced a high-level plot that shows each case, distinguishes between treatment groups, and provides the average value for each experimental group. On the other hand, the generic prompt led it to treat all the women of each year as a case, and it only provided the average calcium level for all women each year. Furthermore, it failed to separate the women into two treatment groups, which makes the plot almost entirely useless. When working with GenAI in the future, it is crucial for myself and others to provide clear, detailed prompts of our goals, needs, and steps. Doing so will cause the AI technology to produce a higher-quality output and to provide more insight into its reasoning.

4 Reflection

4.1 Persistence is Everything!

When I walked into the Pollock building for the first day of my first ever programming class, I came in with a naive view of the field. I envisioned myself hastily typing lines of code as stunning plots and tidy dataframes appeared on my screen. Little did I know that (especially at first) I would receive an error message more often than not. However, my view of those error messages transformed throughout the course. I used to get frustrated whenever I got an error message and get mad at both myself and R. Now, in contrast, I have found that I am better able to roll with the punches and not the let the error messages derail me. I had no idea how important determination and resilience would be when learning my first programming language, and I would like to think that I grew in both of those areas throughout the semester.

4.2 The Importance of Being Detail-Oriented

Another lasting impact that Stat 184 had on me is the importance of being meticulous and detail-oriented, especially when programming. It became clear quickly that this was a goal of the professors when we were given the difficult task of creating a plan for making an origami paper boat. I remember being shocked by how many steps and needs we were supposed to have in our plan! I would like to think that the current version of myself would be able to produce a plan that methodical and detailed. Planning ahead is especially important in programming as the initial time investment always proves to be worth it in the long run. I have found that planning my code in advnance challenged me to reflect on what I learned throughout the course and ponder the best route forward. I am excited to apply this forward-thinking mindset to other areas of life as well.

5 Code Appendices

5.1 Personal Code Appendix

```
# Step 1: Load needed packages----

library(tidyverse)
library(rvest)
library(devtools)
library(dcData)
library(knitr)
library(janitor)
library(kableExtra)
library(plotly)
# Step 2: Scrape airport data----
airportURL <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
airportRawList <- read_html(x = airportURL) |>
  html_elements(css = "table") |>
  html_table()
airportInfo2025 <- airportRawList[[1]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2025) # Add the year as an identifier
airportInfo2024 <- airportRawList[[2]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2024) # Add the year as an identifier
airportInfo2023 <- airportRawList[[3]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2023) # Add the year as an identifier
airportInfo2022 <- airportRawList[[4]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
```

```r
  # Select only the 6 airports we are interested in
  mutate(year = 2022) # Add the year as an identifier
airportInfo2021 <- airportRawList[[5]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2021) # Add the year as an identifier
airportInfo2020 <- airportRawList[[6]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2020) # Add the year as an identifier

#Step 3: Wrangle and join the dataframes----

airportProgress <- bind_rows(airportInfo2025, airportInfo2024,
                             airportInfo2023, airportInfo2022, airportInfo2021, airportInfo2
  # Join the dataframes so that each case is an airport and year
  separate_wider_delim(
    cols = totalpassengers, # Separate the numerical value from the text in brackets
    delim = "[",
    names = c("annualTravelers", "unnecessaryInfo"),
    too_few = "align_start",
    too_many = "merge"
  ) |>
  select(-unnecessaryInfo) |> # Remove the column containing the unnecessary text in brackets
  rename(
    "annual%Change" = "percent_change" # Add a descriptive column name
  ) |>
  separate_wider_delim( # Isolate the percent symbol from the numerical entry
    cols = "annual%Change",
    delim = "%",
    names = c("annualPercentChange", "Junk")) |>
  select(-Junk, -rank) |> # Remove the percent symbols and the Rank column
  mutate(
    annualTravelers = parse_number(annualTravelers),
    # Remove the commas from cells
    annualPercentChange = as.numeric(annualPercentChange),
    # Pass the contents in this column as numeric, not character strings
    annualPercentChange = case_when(
```

```r
      # Add back the negative signs that got dropped during data scraping
      annualPercentChange == 01.6 ~ -01.6,
      annualPercentChange == 8.0 ~ -8.0,
      annualPercentChange == 40.4 ~ -40.4,
      annualPercentChange == 61.2 ~ -61.2,
      annualPercentChange == 63.5 ~ -63.5,
      annualPercentChange == 70.8 ~ -70.8,
      annualPercentChange == 73.4 ~ -73.4,
      .default = annualPercentChange)) |>
  rename(annualRankChange = rankchange) |>
  # Create a more descriptive column name
  mutate(
    annualRankChange = replace_na(annualRankChange, 0,),
    # Replace NA values with 0
    continent = case_when( # Add a column containing each airport's continent
      country == "United States" ~ "North America",
      country == "China" ~ "Asia",
      country == "France" ~ "Europe",
      country == "Germany" ~ "Europe"
    )
  )

airportPercentChange <- airportProgress |>
  group_by(airport) |>
  summarise(# Compute minimum percent change
            minAnnualPercentChange = min(annualPercentChange, na.rm = TRUE),
            # Compute maximum percent change
            maxAnnualPercentChange = max(annualPercentChange, na.rm = TRUE),
            # Compute average percent change
            avgAnnualPercentChange = mean(annualPercentChange, na.rm = TRUE),
            # Compute median percent change
            medianAnnualPercentChange = median(annualPercentChange, na.rm = TRUE),
            # Compute st dev for percent change
            stdevAnnualPercentChange = sd(annualPercentChange, na.rm = TRUE),
            )
airportPercentChange |>
  knitr::kable(
    format = "latex",
    digits = 2, # Truncate values to two decimal places
    format.args = list(big.mark = ','), # Add commas where necessary
    align = 'lccccc', # Align the text
    col.names = c('Airport', 'Min Annual Percent Change',
```

```r
                    'Max Annual Percent Change',
                    'Average Annual Percent Change',
                    'Median Annual Percent Change',
                    'St Dev Annual Percent Change')
    # Add descriptive column names
  ) |>
  kable_classic(
    lightable_options = 'striped'
    # Added light stripes to make the table more visually appealling
  ) |>
  footnote( # Add a footnote describing the data source
    general = 'Data sourced from the Airports Council International via Wikipedia',
    footnote_as_chunk = TRUE # Fit the format on one line
  ) |>
    kableExtra::kable_styling(latex_options = "scale_down")

# Step 1: Load needed packages----

library(tidyverse)
library(rvest)
library(devtools)
library(dcData)
library(knitr)
library(janitor)
library(kableExtra)
library(plotly)
# Step 2: Scrape airport data----
airportURL <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
airportRawList <- read_html(x = airportURL) |>
  html_elements(css = "table") |>
  html_table()
airportInfo2025 <- airportRawList[[1]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2025) # Add the year as an identifier
airportInfo2024 <- airportRawList[[2]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
```

```r
  mutate(year = 2024) # Add the year as an identifier
airportInfo2023 <- airportRawList[[3]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2023) # Add the year as an identifier
airportInfo2022 <- airportRawList[[4]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2022) # Add the year as an identifier
airportInfo2021 <- airportRawList[[5]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2021) # Add the year as an identifier
airportInfo2020 <- airportRawList[[6]] |>
  clean_names() |>
  filter(`code_iata_icao` %in% c("ATL/KATL", "FRA/EDDF", "PKX/ZBAD",
                                 "ORD/KORD", "CDG/LFPG", "CAN/ZGGG")) |>
  # Select only the 6 airports we are interested in
  mutate(year = 2020) # Add the year as an identifier

#Step 3: Wrangle and join the dataframes----

airportProgress <- bind_rows(airportInfo2025, airportInfo2024,
                             airportInfo2023, airportInfo2022, airportInfo2021, airportInfo2
  separate_wider_delim(
    cols = totalpassengers, # Isolate the numerical value
    delim = "[",
    names = c("annualTravelers", "unnecessaryInfo"),
    too_few = "align_start",
    too_many = "merge"
  ) |>
  select(-unnecessaryInfo) |> # Remove unnecessary text
  rename(
    "annual%Change" = "percent_change" # Add a descriptive column name
  ) |>
  separate_wider_delim( # Isolate the percent symbol from the numerical entry
```

```r
    cols = "annual%Change",
    delim = "%",
    names = c("annualPercentChange", "Junk")) |>
  select(-Junk, -rank) |> # Remove the percent symbols and the Rank column
  mutate(
    annualTravelers = parse_number(annualTravelers),
    # Removed the commas from cells
    annualPercentChange = as.numeric(annualPercentChange),
    # Passed the contents in this column as numeric, not character strings
    annualPercentChange = case_when(
      # Added back the negative signs that got dropped during data scraping
      annualPercentChange == 01.6 ~ -01.6,
      annualPercentChange == 8.0 ~ -8.0,
      annualPercentChange == 40.4 ~ -40.4,
      annualPercentChange == 61.2 ~ -61.2,
      annualPercentChange == 63.5 ~ -63.5,
      annualPercentChange == 70.8 ~ -70.8,
      annualPercentChange == 73.4 ~ -73.4,
      .default = annualPercentChange)) |>
  rename(annualRankChange = rankchange) |>
  # Created a more descriptive column name
  mutate(
    annualRankChange = replace_na(annualRankChange, 0,), # Replace NA values with 0
    continent = case_when( # Add a column containing each airport's continent
      country == "United States" ~ "North America",
      country == "China" ~ "Asia",
      country == "France" ~ "Europe",
      country == "Germany" ~ "Europe"
    )
  )

airportPlotData <- airportProgress |>
  mutate(annualTravelers = annualTravelers / 1000000)

ggplot( # Create the plot's frame
  data = airportPlotData,
  mapping = aes( # map year to the x-axis and annual travelers to the y-axis
    x = year,
    y = annualTravelers,
    color = airport, # Color points based on their Airport name
  )
) +
```

```r
  geom_point() + # Create a scatterplot
  geom_line() + # Improvement: Connect the points for each airport
  scale_color_hue(direction = 1) +
  labs( # Improvement: Add descriptive axis titles, labels, and a plot title
    x = "Year",
    y = "Annual Travelers (millions)",
    color = "Airport",
    title = "Annual Travelers for Airports in Asia, Europe, and North America from 2020-2025"
    caption = "Data sourced from the Airports Council International via Wikipedia") +
  facet_wrap(~ continent) + # Create three facets: one for each continent
  theme_bw() + # Select the black and white background for visual appeal
  theme(
    legend.position = "bottom" # Align the legend to the right of the facets
  )


library(tidyverse)
library(patchwork)

coordinateGenerator <- function(xLowBound, xHighBound, yLowBound, yHighBound, sampleSize) {
  # Parameters of the function are above
  xCoord <- runif(sampleSize, min = xLowBound, max = xHighBound)
  # Generated a desired number of random x-values
  yCoord <- runif(sampleSize, min = yLowBound, max = yHighBound)
  # Generated a desired number of random y-values
  orderedPair <- (data.frame(xCoord, yCoord))
  # Display the ordered pairs in a dataframe
  return(orderedPair) # display the dataframe with the ordered pairs
}

# Generate 100 ordered pairs, with the x-values ranging from 0 to 1 and the y-values
# ranging from 0 to 1.5, and display the results in a dataframe
newOrderedPairs <- coordinateGenerator(xLowBound = 0, xHighBound = 1,
                                       yLowBound = 0, yHighBound = 1.5,
                                       sampleSize = 100)
# Apply the beta distribution to all x-coordinates that were generated
betaDistribution <- data.frame(dbeta(newOrderedPairs$xCoord, shape1 = 2, shape2 = 2)) |>
  rename( #Create a more descriptive column title for the output
    "dBetaOutput" = dbeta.newOrderedPairs.xCoord..shape1...2..shape2...2.
  )
coordinatesWithBeta <- bind_cols(newOrderedPairs, betaDistribution) |>
  # Combine the ordered pairs and beta distribution output
```

```r
  mutate(
    flag = case_when( # Create a new flagging system
      yCoord > dBetaOutput ~ "above",
      yCoord <= dBetaOutput ~ "below"
    )) |>
  mutate(
    value = case_when( # Create a numerical flagging system
      flag == "above" ~ 1,
      flag == "below" ~ 0
    )
  )
hundredProportionBelow <- coordinatesWithBeta |>
  summarise(
  pointsBelow = mean(value)
  # This average tells us the proportion of ordered pairs below the curve
)

plot100 <- ggplot( # Create the plot's frame
  data = coordinatesWithBeta,
  mapping = aes(
    x = xCoord,
    y = yCoord,
    color = flag,
    # Color points based on whether they are above or below the function
  )
) +
  stat_function(
  fun = dbeta,
  args = list(shape1 = 2, shape2 = 2), # parameters of dbeta
  xlim = c(0, 1),
  col = "#000000" # Improvement
) +
  geom_point(size = 2) + # Improvement
  labs( # Add descriptive axis titles, labels, and a plot title
    x = "X",
    y = "Y",
    color = "Flag",
    caption = paste("Est. Numerical Integration:", hundredProportionBelow$pointsBelow)
  ) +
  scale_color_manual(
    values = c("#B22222", "#0BDA47")
    # All points above the curve are red, and points below it are green
```

```r
  ) +
  theme_bw() + # Improvement
  theme(
    legend.position = "bottom" # Improvement
  )

#10 points
tenOrderedPairs <- coordinateGenerator(xLowBound = 0, xHighBound = 1,
                                       yLowBound = 0, yHighBound = 1.5,
                                       sampleSize = 10)

tenBetaDistribution <- data.frame(dbeta(tenOrderedPairs$xCoord,
                                        shape1 = 2, shape2 = 2)) |>
  rename(
    "dBetaOutput" = dbeta.tenOrderedPairs.xCoord..shape1...2..shape2...2.
  )
tenWithBeta <- bind_cols(tenOrderedPairs, tenBetaDistribution) |>
  mutate(
    flag = case_when(
      yCoord > dBetaOutput ~ "above",
      yCoord <= dBetaOutput ~ "below"
    )) |>
  mutate(
    value = case_when(
      flag == "above" ~ 1,
      flag == "below" ~ 0
    )
  )
tenProportionBelow <- tenWithBeta |>
  summarise(
    pointsBelow = mean(value)
  )

plot10 <- ggplot(
  data = tenWithBeta,
  mapping = aes(
    x = xCoord,
    y = yCoord,
    color = flag,
  )
) +
  stat_function(
```

```
    fun = dbeta,
    args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1),
    col = "#000000"
  ) +
  geom_point(size = 2) +
  labs(
    x = "X",
    y = "Y",
    color = "Flag",
    caption = paste("Est. Numerical Integration:",
                    tenProportionBelow$pointsBelow)
  ) +
  scale_color_manual(
    values = c("#B22222", "#0BDA47")
  ) +
  theme_bw() +
  theme(
    legend.position = "bottom"
  )
#1000 points
thousandOrderedPairs <- coordinateGenerator(xLowBound = 0,
                                            xHighBound = 1, yLowBound = 0,
                                            yHighBound = 1.5, sampleSize = 1000)

thousandBetaDistribution <- data.frame(dbeta(thousandOrderedPairs$xCoord,
                                             shape1 = 2, shape2 = 2)) |>
  rename(
    "dBetaOutput" = dbeta.thousandOrderedPairs.xCoord..shape1...2..shape2...2.
  )
thousandWithBeta <- bind_cols(thousandOrderedPairs,
                              thousandBetaDistribution) |>
  mutate(
    flag = case_when(
      yCoord > dBetaOutput ~ "above",
      yCoord <= dBetaOutput ~ "below"
    )) |>
  mutate(
    value = case_when(
      flag == "above" ~ 1,
      flag == "below" ~ 0
    )
```

```r
  )
thousandProportionBelow <- thousandWithBeta |>
  summarise(
    pointsBelow = mean(value)
  )

plot1000 <- ggplot(
  data = thousandWithBeta,
  mapping = aes(
    x = xCoord,
    y = yCoord,
    color = flag,
  )
) +
  stat_function(
    fun = dbeta,
    args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1),
    col = "#000000"
  ) +
  geom_point(size = 2) +
  labs(
    x = "X",
    y = "Y",
    color = "Flag",
    caption = paste("Est. Numerical Integration:", thousandProportionBelow$pointsBelow)
  ) +
  scale_color_manual(
    values = c("#B22222", "#0BDA47")
  ) +
  theme_bw() +
  theme(
    legend.position = "bottom"
  )

#10000 points
tenthousOrderedPairs <- coordinateGenerator(xLowBound = 0, xHighBound = 1,
                                            yLowBound = 0, yHighBound = 1.5,
                                            sampleSize = 10000)

tenthousBetaDistribution <- data.frame(dbeta(tenthousOrderedPairs$xCoord,
                                             shape1 = 2, shape2 = 2)) |>
```

```r
  rename(
    "dBetaOutput" = dbeta.tenthousOrderedPairs.xCoord..shape1...2..shape2...2.
  )
tenthousWithBeta <- bind_cols(tenthousOrderedPairs,
                              tenthousBetaDistribution) |>
  mutate(
    flag = case_when(
      yCoord > dBetaOutput ~ "above",
      yCoord <= dBetaOutput ~ "below"
    )) |>
  mutate(
    value = case_when(
      flag == "above" ~ 1,
      flag == "below" ~ 0
    )
  )
tenthousProportionBelow <- tenthousWithBeta |>
  summarise(
    pointsBelow = mean(value)
  )

plot10000 <- ggplot(
  data = tenthousWithBeta,
  mapping = aes(
    x = xCoord,
    y = yCoord,
    color = flag,
  )
) +
  stat_function(
    fun = dbeta,
    args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1),
    col = "#000000"
  ) +
  geom_point(size = 2) +
  labs(
    x = "X",
    y = "Y",
    color = "Flag",
    caption = paste("Est. Numerical Integration:",
                    tenthousProportionBelow$pointsBelow)
```

```r
  ) +
  scale_color_manual(
    values = c("#B22222", "#0BDA47")
  ) +
  theme_bw() +
  theme(
    legend.position = "bottom"
  )
plot10
plot100
plot1000
plot10000


# ==========================
# Calcium Study Project
# ==========================

# --------------------------
# Load packages
# --------------------------
library(tidyverse)

# --------------------------
# Load the calcium csv file (robust import)
# --------------------------
calcium <- read_csv("calcium.csv", show_col_types = FALSE)

# --------------------------
# Fix common CSV issues
# --------------------------

# Remove empty columns (caused by leading commas)
calcium <- calcium %>%
  select(where(~ !all(is.na(.))))

# If column names are incorrect or missing, reset them
if (!"y0" %in% names(calcium)) {
  colnames(calcium) <- c(
    "y0", "y1", "y2", "y3",
    "y0.1", "y1.1", "y2.1", "y3.1"
  )
```

```r
}

# -------------------------
# Add descriptive structure
# -------------------------

# Add ID (each woman = one case)
calcium <- calcium %>%
  mutate(id = row_number())

# Rename columns for clarity
calcium <- calcium %>%
  rename(
    null_y0 = y0,
    null_y1 = y1,
    null_y2 = y2,
    null_y3 = y3,
    exp_y0  = y0.1,
    exp_y1  = y1.1,
    exp_y2  = y2.1,
    exp_y3  = y3.1
  )

# -------------------------
# Rearrange data (tidy format)
# -------------------------

calcium_long <- calcium %>%
  pivot_longer(
    cols = -id,
    names_to = c("treatment_group", "year"),
    names_sep = "_y",
    values_to = "calcium_level"
  )

# -------------------------
# Clean variables
# -------------------------

calcium_long <- calcium_long %>%
  mutate(
    treatment_group = recode(treatment_group,
```

```r
                              "null" = "Control",
                              "exp"  = "Experimental"),
    year = as.numeric(year),
    calcium_level = round(calcium_level, 1)
  )

# =========================
# Create Data Visualization
# =========================

calcium_plot <- ggplot(
  calcium_long,
  aes(
    x = year,
    y = calcium_level,
    color = treatment_group
  )
) +
  geom_point(alpha = 0.7) +                 # scatterplot points
  geom_line(aes(group = id), alpha = 0.3) + # individual trajectories
  stat_summary(                             # group trend line
    fun = mean,
    geom = "line",
    linewidth = 1.2
  ) +
  labs(
    title = "Calcium Levels Over Time by Treatment Group",
    x = "Year of Study",
    y = "Calcium Level",
    color = "Treatment Group",
    caption = "Source: Calcium supplementation study dataset"
  ) +
  theme_minimal()

# -------------------------
# Render the plot
# -------------------------
calcium_plot
```

5.2 GenAI Code Appendix

The following information is for the detailed response provided by ChatGPT:

GenAI Tool Used: ChatGPT Version 5.3

Date Used: April 16th, 2026

Prompts and Responses:

- Prompt:
  - Goal: To wrangle the data from the calcium csv into tidy form and create an informative data visualization that shows how calcium levels between both treatment groups varied over the experiment's duration Needs:
  - Nouns: Plan, R Project, R Script, tidyverse, packages, calcium csv, data, columns, scatterplot, framework, mapping, title, treatment group, caption
  - Verbs: Create, Copy, Fill, Wrangle, Load, Add, Rearrange, Define, Color, Select, Add, Render, Debug, Generate
  - Steps: Create my plan Create an R Project for this work Create a blank R script Copy and paste the data wrangling template Fill in the template with any relevant information Wrangle the data Load necessary packages Load the calcium csv file Add descriptive column names Rearrange the data so that each case is a woman in the study. The data in the 5th through 8th columns should be joined with the 1st through 4th columns so that there is one column for each year's data A new column should be created to identify whether the women were part of the null or experimental groups Each cell should contain the same number of decimal points Create an informative data visualization that shows the difference in calcium levels between the null treatment and experimental groups over the duration of the experiment Define the framework and mapping for the plot Color the points based on their treatment group Select the appropriate geometry Add descriptive titles for the plot axes Add an informative caption that cites the data source Render the plot Debug if any errors came up Generate Alt Text and a long description for the plot
- Response:
  - "Good—this is a very solid plan already. I'll tighten it into something you can directly execute in **R (tidyverse style)**.

    Refined Plan (What you actually need to do)

    Right now your CSV has **two groups side-by-side**:

    * y0–y3 → likely **null/control group**
    * y0.1–y3.1 → likely **experimental group**

Each row = a woman
Each column = calcium level at a specific year

But this is **wide format**, and you need **tidy (long) format**.

Step 1: Set up your R Project + Script

You already outlined this well:

* Create R Project
* Create script (e.g., calcium_analysis.R)
* Load packages:

library(tidyverse)

Step 2: Load the Data

calcium <- read_csv("calcium.csv")

Step 3: Clean + Rename Columns

Right now your columns look like:

y0, y1, y2, y3, y0.1, y1.1, y2.1, y3.1

Rename them so they clearly indicate:

* **year**
* **treatment group**

Step 4: Add an ID Column

Each woman should be identifiable:

calcium <- calcium %>%

mutate(id = row_number())

Step 5: Pivot to Tidy Format

This is the **most important step**.

calcium_long <- calcium %>%

Step 6: Clean Variables

Make them readable:

calcium_long <- calcium_long %>%

Step 7: Create Visualization

You want:

* comparison over time
* grouped by treatment
* clear trend

Best choice: line plot + points

ggplot(calcium_long, aes(x = year, y = calcium, color = treatment)) +

Why This Works (Important Concept)

* pivot_longer() → makes data tidy
* stat_summary(mean) → shows **group trends instead of noisy individual lines**
* color → distinguishes treatment groups clearly

Step 8: Debug Checklist

If something breaks, check:

* Column names not matching → use colnames(calcium)
* y0.1 issues → must use backticks (y0.1)
* plot looks messy → you forgot stat_summary

Step 9: Alt Text

Short alt text:

Line graph showing average calcium levels over four years for control and experimental groups, with the experimental group generally maintaining higher calcium levels.

Step 10: Long Description

This plot displays the average calcium levels of women over four years in a study comparing a control (null) group and an experimental treatment group. The x-axis represents time in years (0 through 3), and the y-axis shows calcium levels. Each group is represented by a distinct colored line with points marking yearly averages. The experimental group consistently shows slightly higher calcium levels than the control group, suggesting a potential positive effect of the treatment over time.

One Small Critique of Your Original Plan

You said:

"Each case is a woman"

That's **true initially**, but after tidying:

Each case becomes **one observation per woman per year**

That distinction matters in stats + visualization."

- Prompt: "What version are you right now?"

  I'm running on the **GPT-5.3-mini** model.

The following information is for the generic response provided by ChatGPT:

GenAI Tool Used: ChatGPT Version 5.3

Date Used: March 31st, 2026

Prompts and Responses:

- Generic Prompt: "Create a data visualization for these data. Calcium.csv"
- Response: I created a visualization showing the **average value of each column** in your dataset.

  What this shows:

  - Each point represents the **mean (average)** of one column (y0, y1, y2, y3, etc.).
  - You can quickly compare which variables tend to have higher or lower values overall.
  - For example, some columns cluster around ~70–73, while others drop closer to ~65.

  If you want something more insightful, I can also create:

  - Boxplots (to show spread and outliers)
  - Time series plots (if rows represent time/order)
  - Correlation heatmaps
  - Side-by-side comparisons (e.g., y0 vs y0.1)

  Just tell me

  Prompt: What is your version information?

  You're chatting with **ChatGPT based on the GPT-5.3 architecture**.

  Key details:

  - **Model**: GPT-5.3
  - **Knowledge cutoff**: August 2025
  - **Current date (system)**: March 31, 2026

If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know